

Practical 2

```
In [1]: import numpy as np

GRID_SIZE = 4
GAMMA = 1.0
THETA = 1e-4

ACTIONS = ['U', 'D', 'L', 'R']
terminal_states = [(0, 0), (3, 3)]

V = np.zeros((GRID_SIZE, GRID_SIZE))
policy = np.ones((GRID_SIZE, GRID_SIZE, len(ACTIONS))) / len(ACTIONS)

def next_state_reward(state, action):
    i, j = state
    if state in terminal_states:
        return state, 0
    if action == 'U': ni, nj = max(i - 1, 0), j
    elif action == 'D': ni, nj = min(i + 1, GRID_SIZE - 1), j
    elif action == 'L': ni, nj = i, max(j - 1, 0)
    elif action == 'R': ni, nj = i, min(j + 1, GRID_SIZE - 1)
    return (ni, nj), -1

def policy_evaluation(V, policy):
    while True:
        delta, new_V = 0, V.copy()
        for i in range(GRID_SIZE):
            for j in range(GRID_SIZE):
                if (i, j) in terminal_states: continue
                v = 0
                for a_idx, action in enumerate(ACTIONS):
                    (ni, nj), reward = next_state_reward((i, j), action)
                    v += policy[i, j, a_idx] * (reward + GAMMA * V[ni, nj])
                new_V[i, j] = v
                delta = max(delta, abs(V[i, j] - new_V[i, j]))
        V = new_V
        if delta < THETA: break
    return V

def policy_improvement(V, policy):
    policy_stable = True
    print("\n--- Final Policy Improvement Summary ---")
    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):
            if (i, j) in terminal_states: continue
            old_action = np.argmax(policy[i, j])
            action_values = np.zeros(len(ACTIONS))
            for a_idx, action in enumerate(ACTIONS):
                (ni, nj), reward = next_state_reward((i, j), action)
                action_values[a_idx] = reward + GAMMA * V[ni, nj]
            best_action = np.argmax(action_values)
            policy[i, j] = np.eye(len(ACTIONS))[best_action]
            if old_action != best_action: policy_stable = False
            print(f"State {i,j}: values={action_values}, best={ACTIONS[best_action]}")
    return policy, policy_stable

# Policy Iteration
while True:
    V = policy_evaluation(V, policy)
    policy, stable = policy_improvement(V, policy)
    if stable: break

print("\nOptimal Value Function:")
print(V)

print("\nOptimal Policy (U,D,L,R):")
for i in range(GRID_SIZE):
```

```

for j in range(GRID_SIZE):
    if (i, j) in terminal_states:
        print("T", end=" ")
    else:
        print(ACTIONS[np.argmax(policy[i, j])], end=" ")
print()

```

--- Final Policy Improvement Summary ---

```

State (0, 1): values=[-14.99893866 -18.99861452 -1.          -20.99842728], best=L
State (0, 2): values=[-20.99842728 -20.9984378  -14.99893866 -22.99824003], best=L
State (0, 3): values=[-22.99824003 -20.99842728 -20.99842728 -22.99824003], best=D
State (1, 0): values=[ -1.          -20.99842728 -14.99893866 -18.99861452], best=U
State (1, 1): values=[-14.99893866 -20.9984378  -14.99893866 -20.9984378 ], best=L
State (1, 2): values=[-20.99842728 -18.99861452 -18.99861452 -20.99842728], best=D
State (1, 3): values=[-22.99824003 -14.99893866 -20.9984378  -20.99842728], best=D
State (2, 0): values=[-14.99893866 -22.99824003 -20.99842728 -20.9984378 ], best=U
State (2, 1): values=[-18.99861452 -20.99842728 -20.99842728 -18.99861452], best=U
State (2, 2): values=[-20.9984378  -14.99893866 -20.9984378  -14.99893866], best=R
State (2, 3): values=[-20.99842728 -1.          -18.99861452 -14.99893866], best=D
State (3, 0): values=[-20.99842728 -22.99824003 -22.99824003 -20.99842728], best=U
State (3, 1): values=[-20.9984378  -20.99842728 -22.99824003 -14.99893866], best=R
State (3, 2): values=[-18.99861452 -14.99893866 -20.99842728 -1.          ], best=R

```

--- Final Policy Improvement Summary ---

```

State (0, 1): values=[-2. -3. -1. -3.], best=L
State (0, 2): values=[-3. -4. -2. -4.], best=L
State (0, 3): values=[-4. -3. -3. -4.], best=D
State (1, 0): values=[-1. -3. -2. -3.], best=U
State (1, 1): values=[-2. -4. -2. -4.], best=U
State (1, 2): values=[-3. -3. -3. -3.], best=U
State (1, 3): values=[-4. -2. -4. -3.], best=D
State (2, 0): values=[-2. -4. -3. -4.], best=U
State (2, 1): values=[-3. -3. -3. -3.], best=U
State (2, 2): values=[-4. -2. -4. -2.], best=D
State (2, 3): values=[-3. -1. -3. -2.], best=D
State (3, 0): values=[-3. -4. -4. -3.], best=U
State (3, 1): values=[-4. -3. -4. -2.], best=R
State (3, 2): values=[-3. -2. -3. -1.], best=R

```

--- Final Policy Improvement Summary ---

```

State (0, 1): values=[-2. -3. -1. -3.], best=L
State (0, 2): values=[-3. -4. -2. -4.], best=L
State (0, 3): values=[-4. -3. -3. -4.], best=D
State (1, 0): values=[-1. -3. -2. -3.], best=U
State (1, 1): values=[-2. -4. -2. -4.], best=U
State (1, 2): values=[-3. -3. -3. -3.], best=U
State (1, 3): values=[-4. -2. -4. -3.], best=D
State (2, 0): values=[-2. -4. -3. -4.], best=U
State (2, 1): values=[-3. -3. -3. -3.], best=U
State (2, 2): values=[-4. -2. -4. -2.], best=D
State (2, 3): values=[-3. -1. -3. -2.], best=D
State (3, 0): values=[-3. -4. -4. -3.], best=U
State (3, 1): values=[-4. -3. -4. -2.], best=R
State (3, 2): values=[-3. -2. -3. -1.], best=R

```

Optimal Value Function:

```

[[ 0. -1. -2. -3.]
 [-1. -2. -3. -2.]
 [-2. -3. -2. -1.]
 [-3. -2. -1.  0.]]

```

Optimal Policy (U,D,L,R):

```

T L L D
U U U D
U U D D
U R R T

```